AgentGuard

Judge Q&A; Prep — how it was built, step by step, and what to say when they ask.

AngelHack: Building with IBM Bob • Team CodeBase Loops • Juan Serrano

The 60-second pitch

I built AgentGuard because AI coding agents get fooled by prompt injection and nobody guards the tool calls. It's a deny-by-default policy engine: every tool call needs an HMAC-signed capability token before it runs. No valid token, no execution — attacks fail closed, and every decision lands in an append-only audit ledger. During this hackathon I rebuilt it around IBM Bob: Bob understands the codebase, generates the security policies, and authored the test suite.

How I built it — step by step

1. The problem.

AI agents execute tool calls on behalf of users. A single injected instruction can turn a helpful agent into one that exfiltrates data or runs destructive commands. System prompts don't stop this — the check has to happen at execution time.

2. The core idea.

Deny by default. Before any tool call runs, the agent must present a capability token — an HMAC-signed, short-lived, scope-limited credential. The policy engine verifies the signature, the expiry, and the scopes, then checks the call against the policy. Anything not explicitly allowed is denied.

3. The policy engine.

Python. A small, readable engine: policies are data, decisions are deterministic, and every decision is recorded. No ML black box in the enforcement path — the guard itself has to be auditable.

4. The audit ledger.

Append-only JSONL. Every allow and deny lands there with the token identity, the tool, the arguments, and the reason. You can replay exactly what an agent did and why.

5. Live scanning.

Secret scanner with real detection patterns plus live dependency scanning (pip-audit / npm audit), so the demo catches leaked keys and vulnerable packages, not just policy violations.

6. Bringing in IBM Bob.

During the hackathon I wired Bob in through MCP: Bob reads the codebase, generates security policies from what it understands, and authored the test suite — 90 tests passing. Bob is a builder on this project, not a mascot.

7. The demo.

A live serverless demo with token login. Judges enter as guests in one click — no signup — and get a skills console: policy checks, token minting and verification, secret scanning, audit queries, and a scripted agent run that shows allows and denies happening in real time.

8. Hardening and opening up.

I tested it like an attacker: sandbox escapes, token tampering, scope escalation, path traversal. Every attack I tried is denied and logged. Then I exposed every capability as a live skill through the Skills API and shipped a zero-dependency Java SDK — the test suite, 90 passing, proves it all on every run.

What the judges will ask — and what to say

Gold = the judge's question. White = your drafted answer. Keep answers short; offer the live demo when a question is easier shown than told.

Q1. What does AgentGuard actually do?

It stands between an AI agent and its tools: every call must carry a valid capability token, and the policy engine decides allow or deny before anything executes. Attacks fail closed.

Q2. How is this different from a good system prompt?

A system prompt is a suggestion the model can be talked out of. AgentGuard is enforcement outside the model: even if the agent is fully compromised by injected instructions, it can't act without a valid token for that exact tool and scope. The trust boundary moved from the model's judgment to cryptography.

Q3. Walk me through the HMAC capability tokens.

Each token is HMAC-SHA256 signed with a server-side secret, carries the subject, scopes, and expiry, and is verified on every request — signature, expiry, and scope, in that order. Tokens are short-lived: guest tokens last an hour, demo logins eight. A tampered token fails signature verification and is denied. Scope escalation is rejected — you can only mint tokens within the scopes you already hold.

Q4. What did IBM Bob actually do on this project?

Three concrete things. First, Bob reads the codebase through the MCP server I built — check_policy, scan_codebase, audit_summary — so it understands the code it's working with. Second, Bob generates the security policies from that understanding. Third, Bob authored the test suite: 90 tests passing, covering the policy engine, tokens, the scanner, the skills API, the trap, and attack cases. Bob is in the build loop, not the slide deck.

Q5. Show me it stopping an attack.

Open the demo, enter as guest, and run a policy check for read_file on /etc/passwd — denied, path escapes sandbox. Try run_command at all — denied, deny-by-default. Then run the scripted agent demo: it shows a legit read allowed, a sandbox escape denied, a secret leak flagged, and a destructive command denied, with every step written to the audit ledger.

Q6. Doesn't this add latency to every tool call?

A local HMAC verification plus a policy lookup — microseconds, no network round trip. Expensive work like dependency scanning happens out of band. Against model inference and tool execution time, the guard is noise.

Q7. What if someone steals a valid token?

Tokens are short-lived and narrowly scoped, so a stolen token is a small, expiring window — and every use is logged against the token identity, so misuse is visible immediately. That's the point of capabilities over ambient authority: the blast radius is the token's scopes, not the agent's everything.

Q8. How do I know the audit ledger wasn't tampered with?

It's append-only JSONL — entries are only ever added, never modified, and each decision records the token identity, tool, arguments, and reason. You can replay the full history of what an agent did and why each call was allowed or denied.

Q9. Why HMAC instead of, say, JWT with RSA?

One service signs and verifies — no key distribution problem, fast, simple to audit. The threat is token forgery by outsiders, and a server-side secret handles that. RSA adds complexity without security here.

Q10. What was the hardest bug you hit?

The skills API's policy-check endpoint initially verified token scopes but skipped the sandbox path check — so it allowed /etc/passwd while the main endpoint denied it. I caught it by testing the demo like an attacker, fixed it to run the identical validation as the main path, and re-verified live. The test suite now covers it.

Q11. Can this work outside Python?

Yes — I shipped a Java SDK with zero dependencies that mints and verifies the exact same tokens, proven with cross-language tests: Java-minted tokens verify in Python and vice versa. The token format is just canonical JSON plus HMAC, so any language can join.

Q12. What is the Skills API?

Every guardrail is a callable skill: policy-check, token-mint, token-verify, secret-scan, audit-query, harness-run, and trap. The demo has a try-it console — pick a skill, send JSON, watch the decision live. It's the product's API and its demo in one; judges can run attacks themselves instead of taking my word.

Q13. What is the trap skill?

A honeypot built into the guard. Every denied call is fingerprinted by technique — traversal, sandbox escape, token tampering, scope escalation, prompt injection — logged to the audit ledger with secrets redacted, and turned into a drafted policy rule for that attack class. Deny-first: the deny happens before and independent of intel capture, so an intel failure can never flip a deny into an allow.

Q14. How do you know the guard actually holds?

I attack it myself: a red-team harness mutates the trap's learned fingerprints into new variants and replays them against the live demo — 76 variants, 74 denied, 0 allowed. The loop is real: the trap fingerprinted the replays as new intel. Every attack the guard survives makes it smarter.

Q15. Why a Java SDK?

Because agents aren't only Python. The SDK has zero dependencies, speaks the exact same HMAC token format, and I proved interop in both directions with real tests. It shows the design is a protocol, not a Python trick — any stack can enforce the same guardrails.

Q16. What's next after the hackathon?

Policy generation fully driven by Bob from natural-language intent; a hosted multi-tenant version so teams can guard agents without self-hosting; SDKs beyond Python and Java. The core — deny-by-default with capability tokens — doesn't change.

Demo cheat sheet — what to click

• **Enter as guest** — one click, no signup. Do this first; it mints a 1-hour token.

• **Policy check:** read_file on *welcome.txt* → ALLOW. Same tool on */etc/passwd* → DENY.

• **Secret scan:** paste text containing an AWS-style key → flagged with its pattern kind.

• **Harness run:** the scripted agent — watch allow/deny decisions stream with reasons.

• **Skills console:** pick any of the seven skills and send JSON — mint a token, verify it, then tamper with one character and watch it fail. Try the **trap** on /etc/passwd and see the fingerprint.

• **Audit query:** every action above is already in the ledger. Show them the trail.

Deny by default, prove every allow, log everything — with IBM Bob in the build loop.